



HOSPITAL MANAGEMENT SYSTEM

DATABASE SYSTEMS

SUBMITTED TO
DR. IRFAN HAMEED

MUHAMMAD ALI QADRI
DEPARTMENT OF COMPUTER & INFORMATION SCIENCES

TABLE OF CONTENTS

0. INTRODUCTION	1
0.1 PROBLEM STATEMENT.....	1
0.2 PROJECT SCOPE	1
1. ENTERPRISE DATA MODEL (ED MODEL)	2
2. ENTITY RELATIONSHIP DIAGRAM	3
2.1 SEPARATE ER DIAGRAMS WITH ATTRIBUTES	3
2.1.1 PATIENT	3
2.1.2 EMPLOYEE	3
2.1.3 APPOINTMENT	4
2.1.4 ADMISSION.....	4
2.1.5 ROOMS	5
2.1.6 TEST	5
2.1.7 INVOICE	6
2.1.8 PATIENT_TESTS	6
2.2 ER DIAGRAMS FOR RELATIONSHIPS	7
2.2.1 PATIENT AND INVOICE.....	7
2.2.2 PATIENT AND TEST.....	7
2.2.3 PATIENT AND APPOINTMENT.....	8
2.2.4 ADMITTED_PATIENT AND ADMISSION.....	8
2.2.5 EMPLOYEE AND APPOINTMENT.....	9
2.2.6 APPOINTMENT AND ADMISSION	9
2.2.7 EMPLOYEE AND ROOMS	10
2.2.8 ADMISSION AND ROOMS.....	10
2.3 BUSINESS RULES.....	11
2.4 COMPLETE ENTITY RELATIONSHIP DIAGRAM	12
3. RELATIONAL SCHEMA.....	13
3.1 FOR EACH ENTITY	13
3.1.1 PATIENT	13
3.1.2 PATIENT_TESTS	13
3.1.3 EMPLOYEE	13
3.1.4 APPOINTMENT	13

3.1.5 ADMISSION.....	14
3.1.6 ROOMS	14
3.1.7 TEST	14
3.1.8 INVOICE	14
3.2 FOR EACH RELATIONSHIP	15
3.2.1 PATIENT AND INVOICE.....	15
3.2.2 PATIENT AND TEST.....	15
3.2.3 PATIENT AND APPOINTMENT.....	16
3.2.4 ADMITTED_PATIENT AND ADMISSION.....	16
3.2.5 EMPLOYEE AND APPOINTMENT.....	16
3.2.6 APPOINTMENT AND ADMISSION	17
3.2.7 EMPLOYEE AND ROOMS	17
3.2.8 ADMISSION AND ROOMS.....	17
3.3 COMPLETE RELATIONAL SCHEMA	18
4. NORMALIZATION	19
5. SAMPLE DATA	19
6. SAMPLE QUERIES	21
6.1 GET EMPLOYEE INFORMATION	21
6.2 GET APPOINTMENTS FOR A SPECIFIC PATIENT	21
6.3 GET ALL TESTS FOR A PATIENT	21
6.4 GET ALL EMPLOYEES IN A DEPARTMENT	22
6.5 GET PATIENT INVOICE.....	22
6.6 GET ALL PRESCRIPTIONS GIVEN BY A DOCTOR	22
7 TABLE CREATION CODE/TRIGGERS/ROLES	23
7.1 TABLE CREATION.....	23
7.2 TRIGGERS.....	29
7.3 ROLES CREATION	34
8. APPROXIMATED STORAGE.....	36

0. INTRODUCTION

This project is a comprehensive **Hospital Management System** designed to efficiently manage hospital operations, including patient care, employee management, room allocations, appointments, and medical testing. It features an extensive database schema comprising tables for patients, employees, rooms, tests, admissions, and invoices, interconnected through foreign keys for data integrity. The system includes triggers to automate processes, such as managing room capacity, auto-inserting related records, and ensuring doctor-specific appointments. Additionally, roles and views are created to enforce security and access control, providing different permissions to roles like doctors, patients, and receptionists. This structured system ensures streamlined workflows, accurate record-keeping, and efficient resource management in a hospital setting.

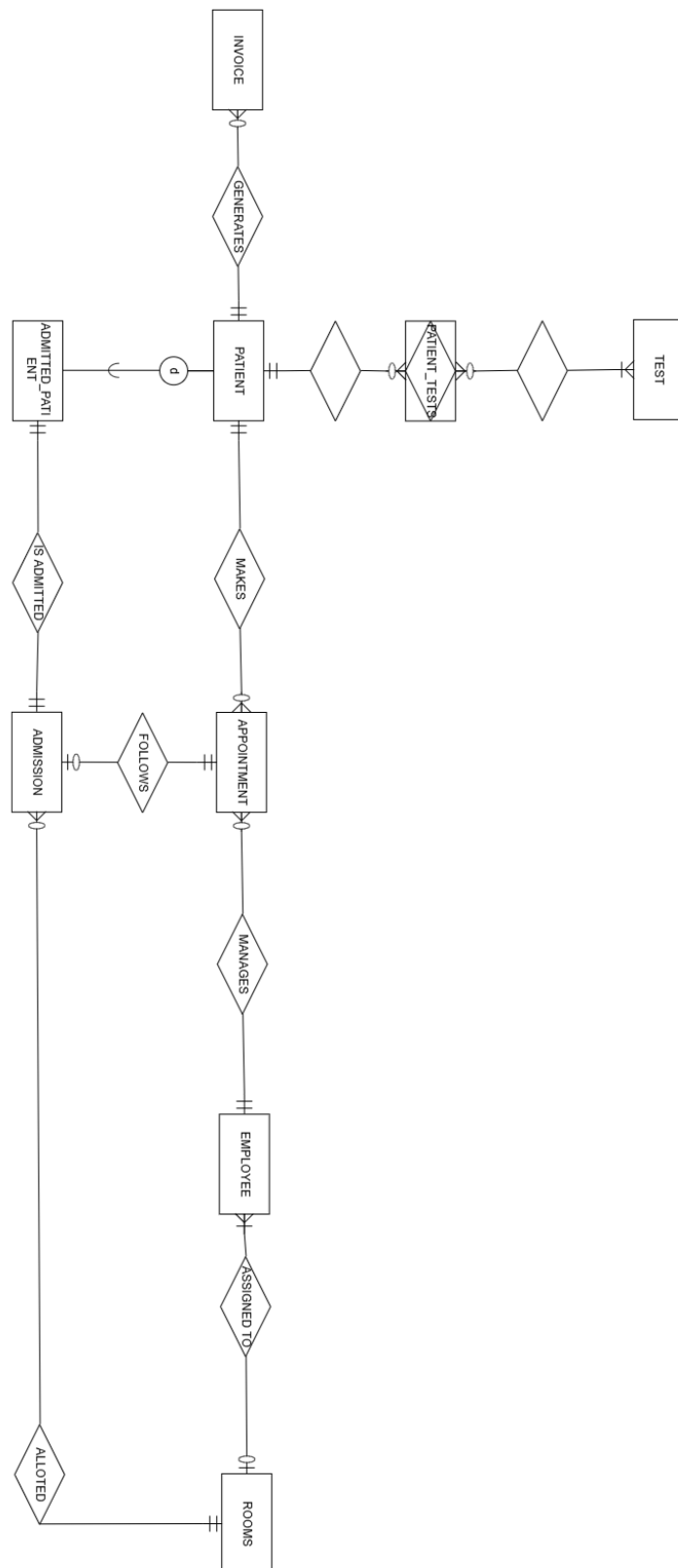
0.1 PROBLEM STATEMENT

The healthcare industry faces challenges in managing patient records, appointments, hospital staff, room allocations, and billing processes efficiently. Traditional manual systems are prone to errors, data loss, and inefficiencies, leading to mismanagement of resources, patient dissatisfaction, and increased operational costs. Hospitals require a robust, automated system to streamline these critical processes, ensure data accuracy, manage room availability, and automate repetitive tasks such as billing and resource allocation. A lack of role-based access control also compromises data security and confidentiality, further emphasizing the need for a comprehensive and secure solution.

0.2 PROJECT SCOPE

The **Hospital Management System (HMS)** will streamline hospital operations by managing patient records, staff information, room allocations, and billing processes. Key features include patient admissions, appointment scheduling, room availability tracking, automated billing, and role-based access control for secure data management. The system ensures data accuracy, operational efficiency, and improved resource utilization.

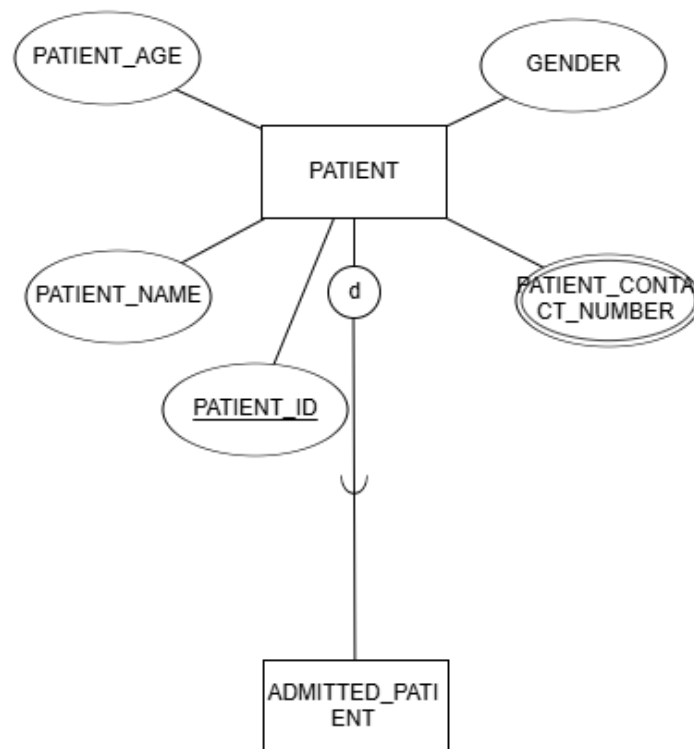
1. ENTERPRISE DATA MODEL (ED MODEL)



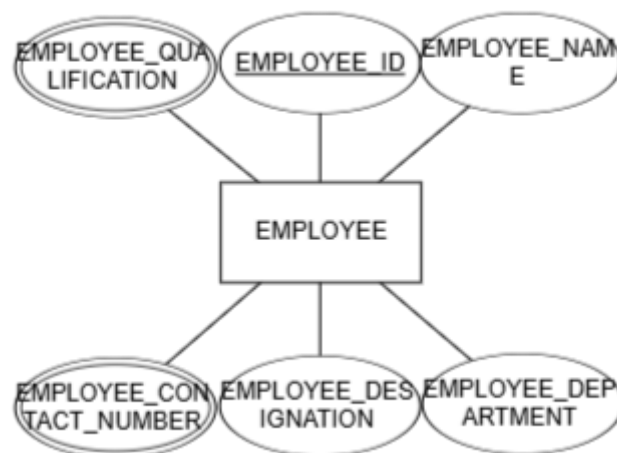
2. ENTITY RELATIONSHIP DIAGRAM

2.1 SEPARATE ER DIAGRAMS WITH ATTRIBUTES

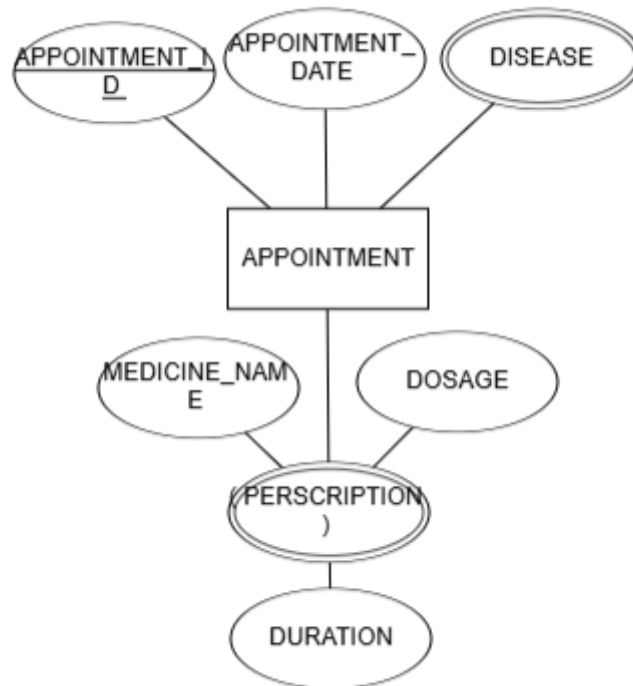
2.1.1 PATIENT



2.1.2 EMPLOYEE



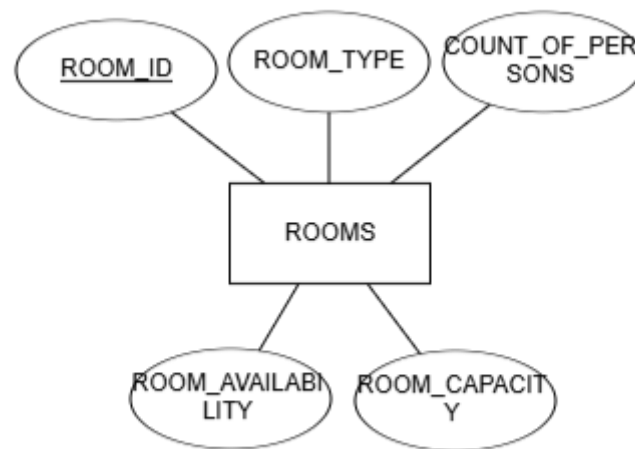
2.1.3 APPOINTMENT



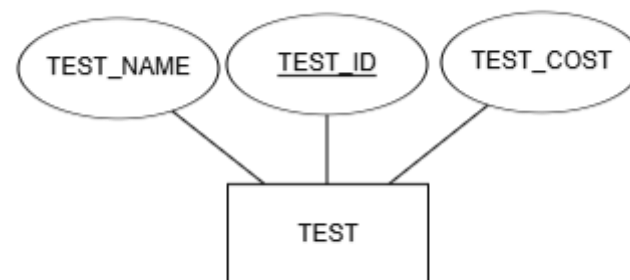
2.1.4 ADMISSION



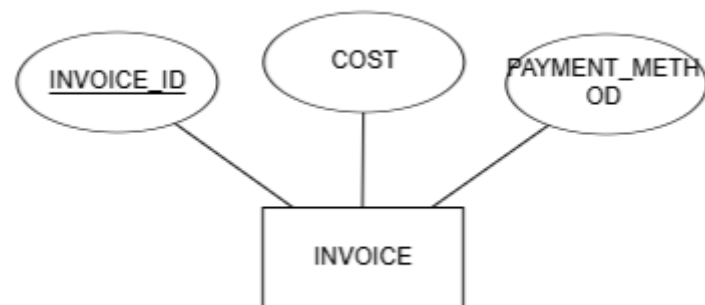
2.1.5 ROOMS



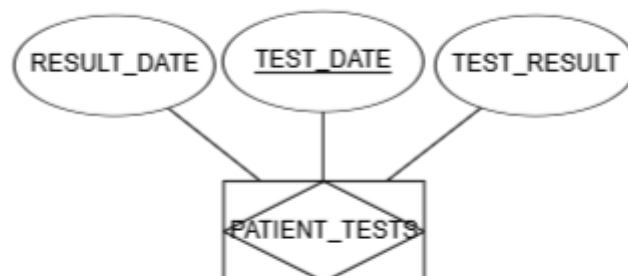
2.1.6 TEST



2.1.7 INVOICE



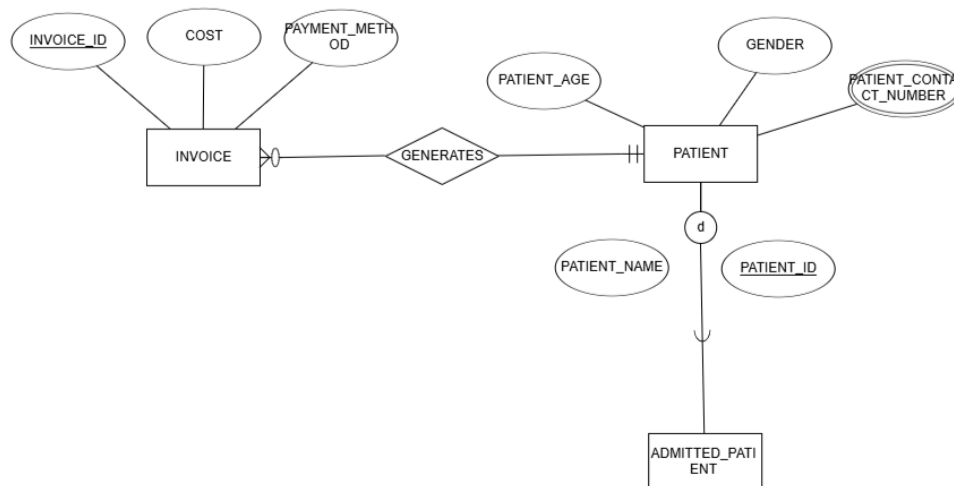
2.1.8 PATIENT_TESTS



2.2 ER DIAGRAMS FOR RELATIONSHIPS

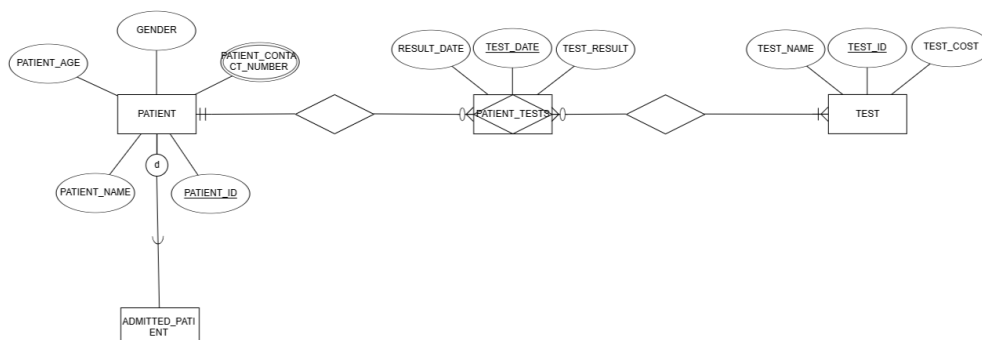
2.2.1 PATIENT AND INVOICE

This illustrates the relationship where an INVOICE is generated for a PATIENT based on their PATIENT_ID and includes details such as COST, PAYMENT_METHOD. The PATIENT entity is characterized by attributes like PATIENT_NAME, PATIENT_AGE, and GENDER, and is linked to the ADMITTED_PATIENT relationship, indicating that the invoice corresponds to a patient who has been admitted for treatment.



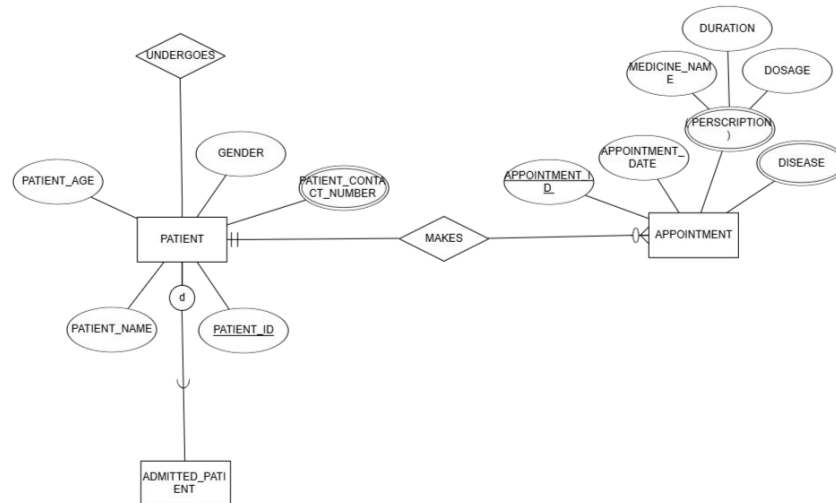
2.2.2 PATIENT AND TEST

It features three main entities: PATIENT, TEST, and an associative entity PATIENT_TESTS that connects patients to tests. The PATIENT entity includes attributes like PATIENT_ID (a unique identifier), PATIENT_NAME, PATIENT_AGE, GENDER, PATIENT_CONTACT_NUMBER, and ADMITTED_PATIENT. The TEST entity includes TEST_ID (a unique identifier), TEST_NAME, and TEST_COST. The associative entity PATIENT_TESTS links PATIENT and TEST with additional attributes such as TEST_DATE, RESULT_DATE, and TEST_RESULT. The relationships include a one-to-many relationship where a patient can take multiple tests, and a test can be performed on multiple patients, forming a many-to-many relationship through the PATIENT_TESTS entity.



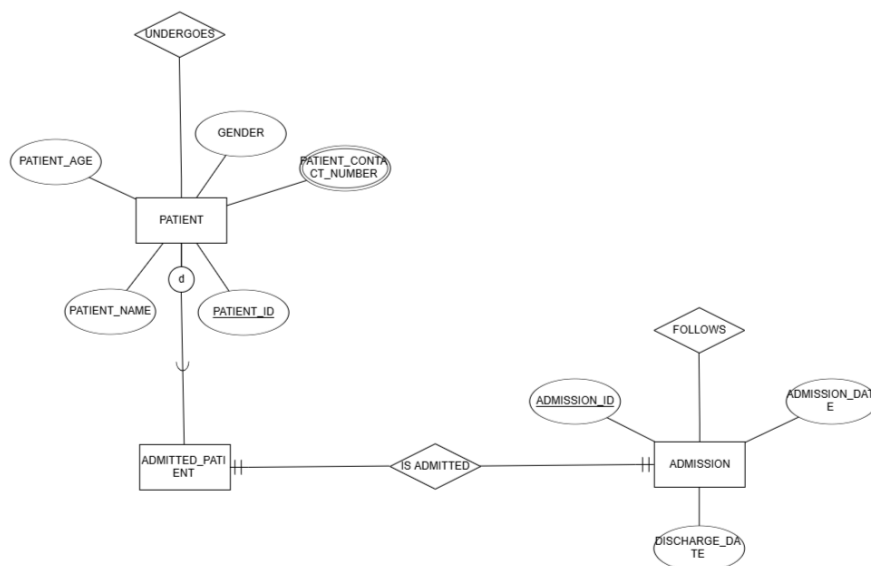
2.2.3 PATIENT AND APPOINTMENT

This illustrates the relationship where a PATIENT makes an APPOINTMENT. The PATIENT entity is identified by PATIENT_ID and includes attributes such as PATIENT_NAME, PATIENT_AGE, GENDER, and PATIENT_CONTACT_NUMBER. The APPOINTMENT is characterized by attributes like APPOINTMENT_ID, APPOINTMENT_DATE, DISEASE, and includes details about the prescribed MEDICINE_NAME, DOSAGE, and DURATION. The PATIENT is also linked to the ADMITTED_PATIENT relationship, indicating that the patient is admitted after the appointment process.



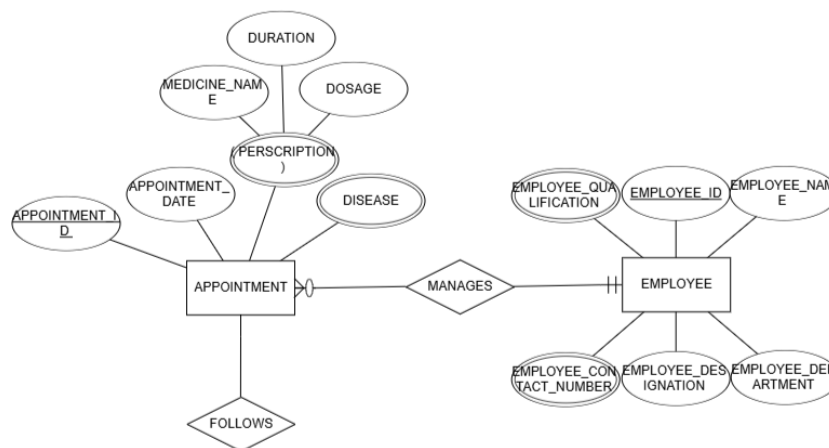
2.2.4 ADMITTED_PATIENT AND ADMISSION

This illustrates the relationship where a PATIENT undergoes an admission process. The ADMITTED_PATIENT relationship signifies that the patient has been admitted, identified by ADMISSION_ID. The ADMISSION entity includes attributes like ADMISSION_DATE and DISCHARGE_DATE, detailing the patient's admission timeline.



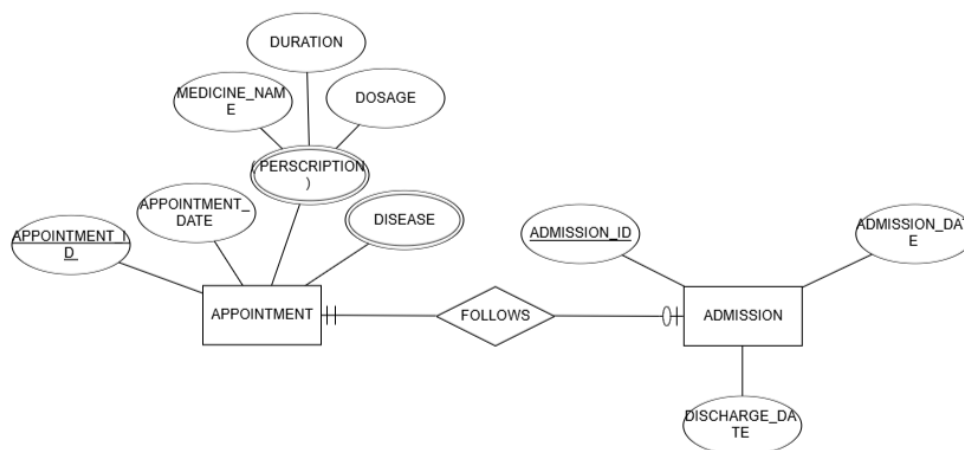
2.2.5 EMPLOYEE AND APPOINTMENT

This illustrates the relationships between employee and appointment. It identifies the appointment by APPOINTMENT_ID and includes attributes such as APPOINTMENT_DATE and DISEASE. The prescription is associated with the appointment and characterized by attributes like MEDICINE_NAME, DOSAGE, and DURATION. Additionally, it represents the doctor with EMPLOYEE_ID, involved in managing appointments, identified by attributes such as EMPLOYEE_NAME, EMPLOYEE_QUALIFICATION, EMPLOYEE_DESIGNATION, and EMPLOYEE_CONTACT_NUMBER. Overall, the diagram shows how appointments lead to prescriptions and how employees manage these processes within the healthcare system.



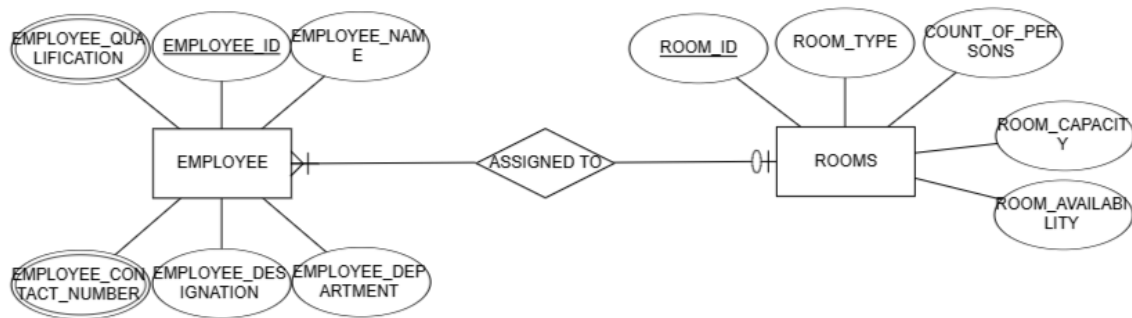
2.2.6 APPOINTMENT AND ADMISSION

This illustrates the relationship between APPOINTMENT, PRESCRIPTION, ADMISSION, AND DISEASE. The Appointment entity contains attributes like Appointment_ID and Appointment_Date and is associated with the Prescription, which includes Medicine_Name, Dosage, and Duration. The relationship indicates that a disease is followed through prescriptions issued during an appointment. The Admission entity, with attributes like Admission_ID, Admission_Date, and Discharge_Date, is linked to the Disease entity, representing cases where patients requiring hospitalization are admitted.



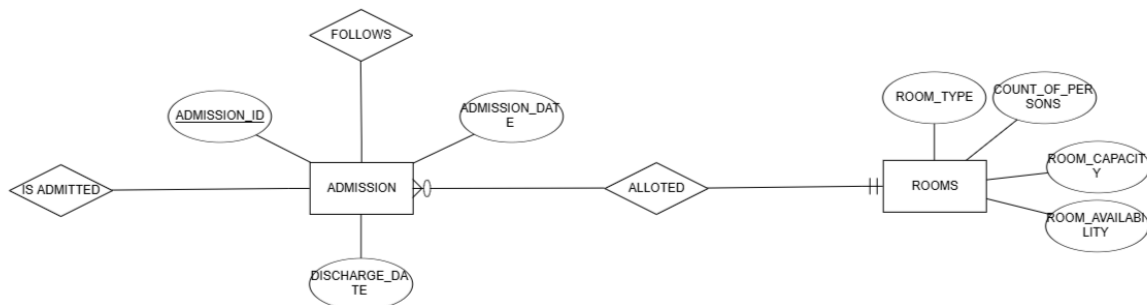
2.2.7 EMPLOYEE AND ROOMS

This illustrates the relationship between the employee AND rooms. It identifies employees by EMPLOYEE_ID and includes attributes such as EMPLOYEE_NAME, EMPLOYEE_QUALIFICATION, EMPLOYEE_DESIGNATION, and EMPLOYEE_CONTACT_NUMBER. The rooms are characterized by ROOM_ID and include attributes like ROOM_TYPE, COUNT_OF_PERSONS, ROOM_CAPACITY, and ROOM_AVAILABILITY. The diagram shows that employees are assigned to rooms, highlighting the interaction between healthcare staff and the facilities they manage.



2.2.8 ADMISSION AND ROOMS

This illustrates the relationship between admission and room entities. It identifies admissions by ADMISSION_ID and includes attributes such as ADMISSION_DATE and DISCHARGE_DATE. The diagram indicates that an admission is followed by an allocation to a room, which is characterized by attributes like ROOM_TYPE, COUNT_OF_PERSONS, ROOM_CAPACITY, and ROOM_AVAILABILITY.



2.3 BUSINESS RULES

Patient and Invoice: A patient has the ability to generate multiple invoices throughout their course of treatment or medical visits. However, each invoice is tied to exactly one patient, ensuring that all charges are associated with a specific individual.

Patient and Appointment: A patient may schedule multiple appointments for various reasons. Each appointment, however, is uniquely linked to one patient, ensuring that the records of each visit are properly attributed.

Appointment and Admission: Every admission is associated with a preceding appointment, meaning that an appointment can result in an admission. While appointments are not always followed by admissions, they serve as the necessary precursor when an admission is required.

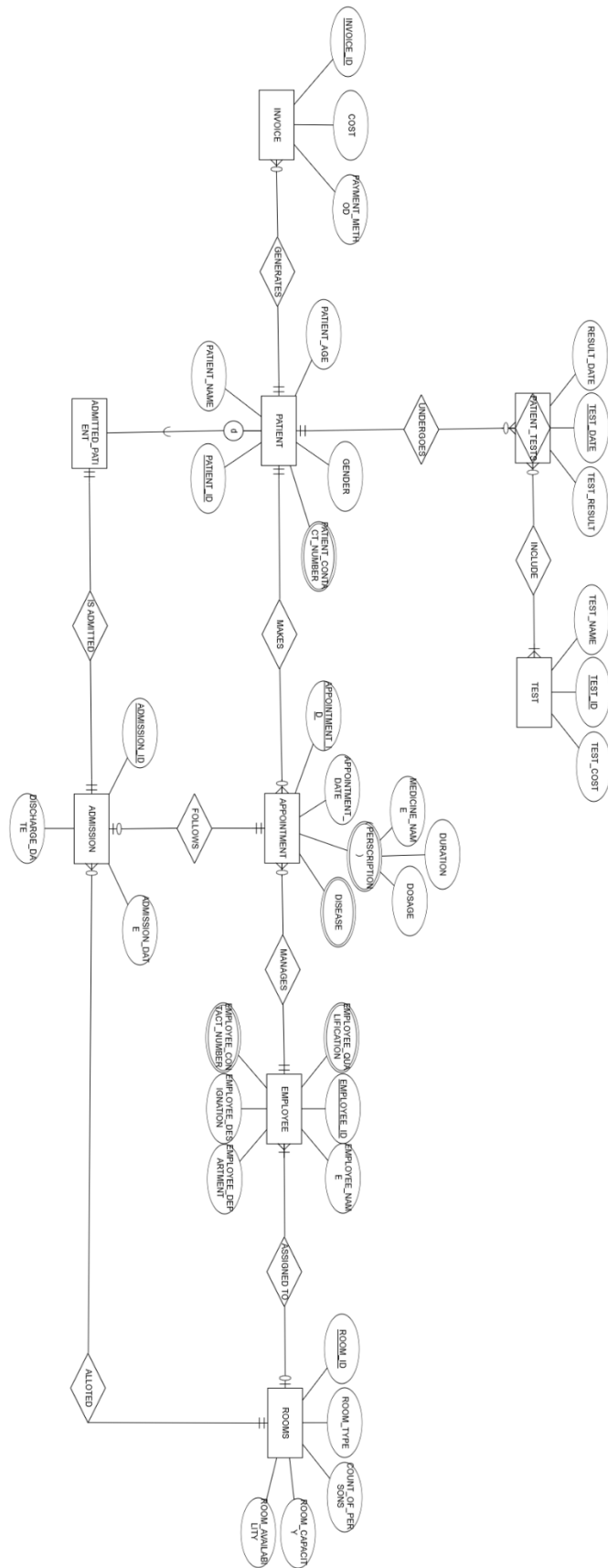
Employee and Rooms: An employee can be assigned to a specific room, and each room may accommodate several employees. This reflects the flexibility of room usage in medical or clinical settings, where multiple staff members may use the same space during their shifts.

Employee and Appointment: An employee, particularly in the role of a healthcare provider or administrative staff, may manage multiple appointments.

Admission and Rooms: A room must be allocated to a specific admission, ensuring that patients admitted to the facility are assigned appropriate accommodations. A single room can be allocated to multiple admissions, particularly in the case of shared or multi-bed rooms.

Patient and Tests: A patient can undergo a variety of tests during their treatment process, and multiple tests can be included in the list of a patient's medical procedures. This relationship ensures comprehensive tracking of the diagnostic procedures a patient undergoes, with tests being clearly linked to the individual patient's medical records.

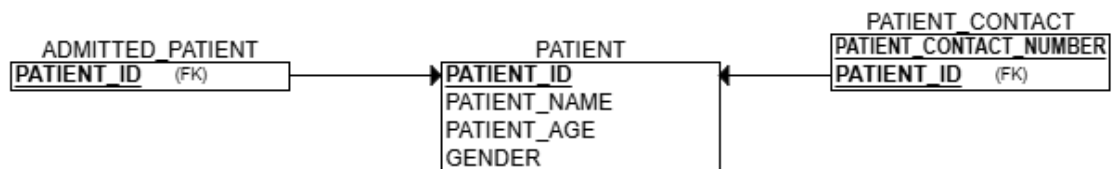
2.4 COMPLETE ENTITY RELATIONSHIP DIAGRAM



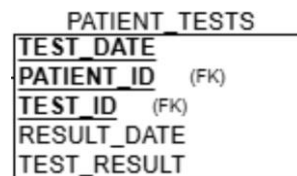
3. RELATIONAL SCHEMA

3.1 FOR EACH ENTITY

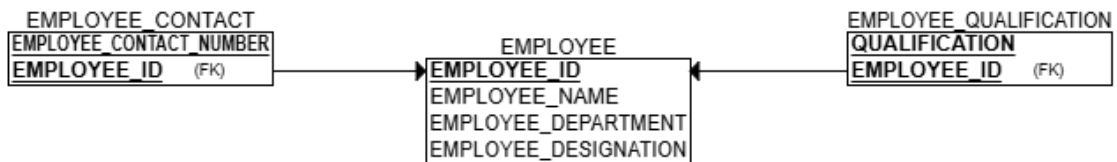
3.1.1 PATIENT



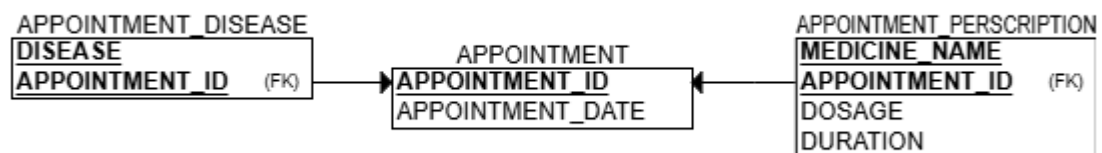
3.1.2 PATIENT_TESTS



3.1.3 EMPLOYEE



3.1.4 APPOINTMENT



3.1.5 ADMISSION

ADMISSION	
<u>ADMISSION_ID</u>	
ADMISSION_DATE	
DISCHARGE_DATE	

3.1.6 ROOMS

ROOMS	
<u>ROOM_ID</u>	
ROOM_TYPE	
COUNT_OF_PERSONS	
ROOM_CAPACITY	
ROOM_AVAILABILITY	

3.1.7 TEST

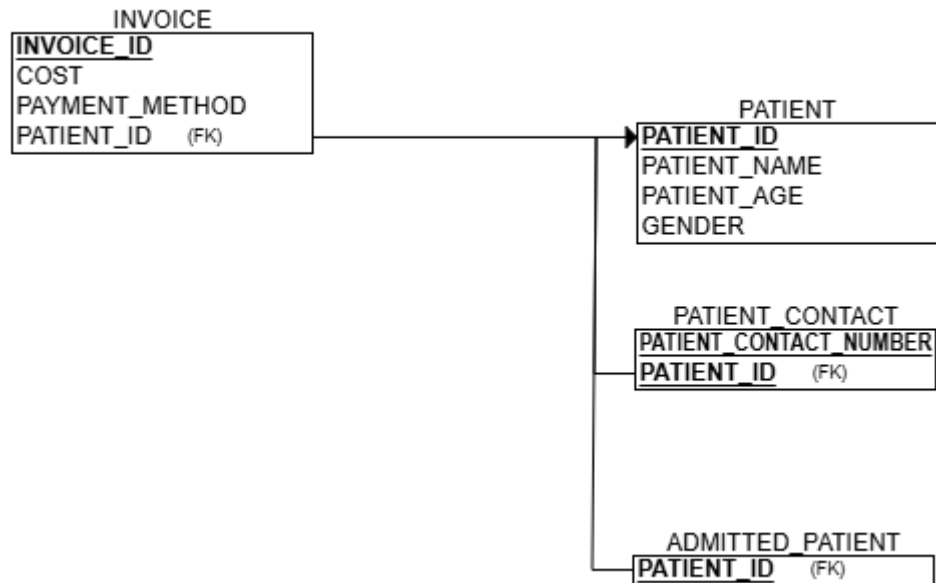
TEST	
<u>TEST_ID</u>	
TEST_NAME	
TEST_COST	

3.1.8 INVOICE

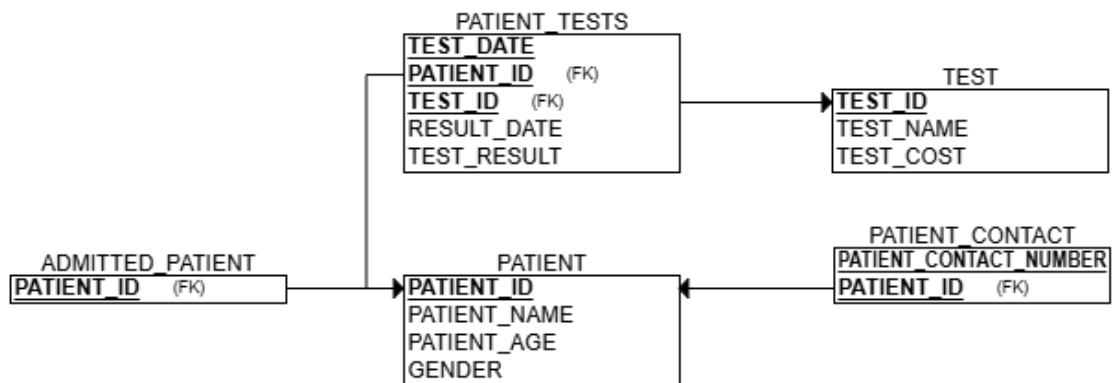
INVOICE	
<u>INVOICE_ID</u>	
COST	
PAYMENT_METHOD	

3.2 FOR EACH RELATIONSHIP

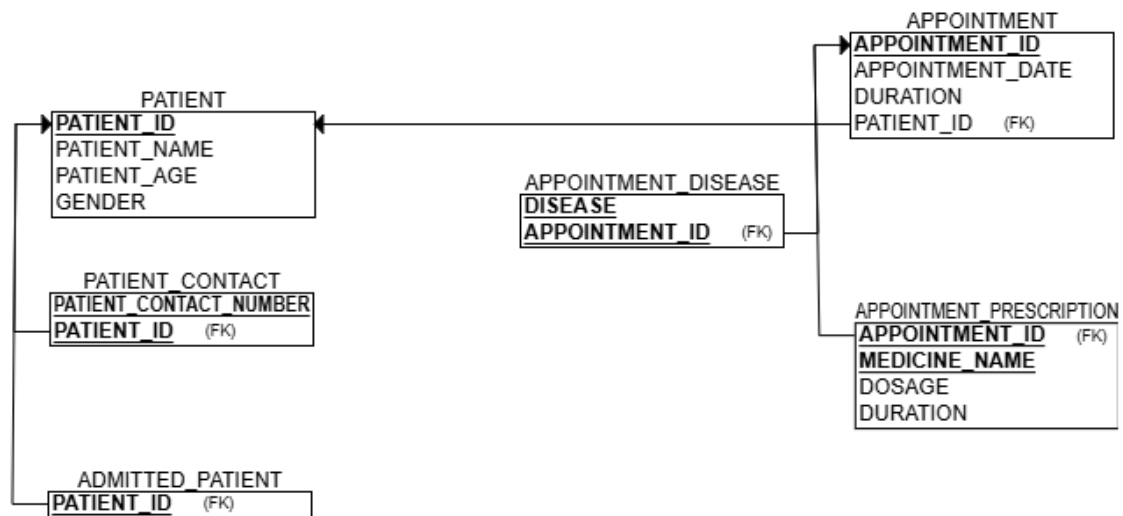
3.2.1 PATIENT AND INVOICE



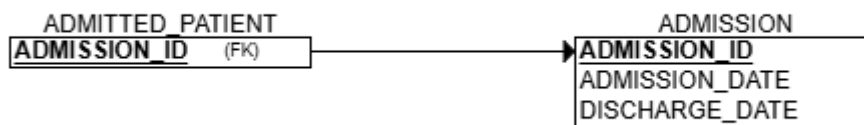
3.2.2 PATIENT AND TEST



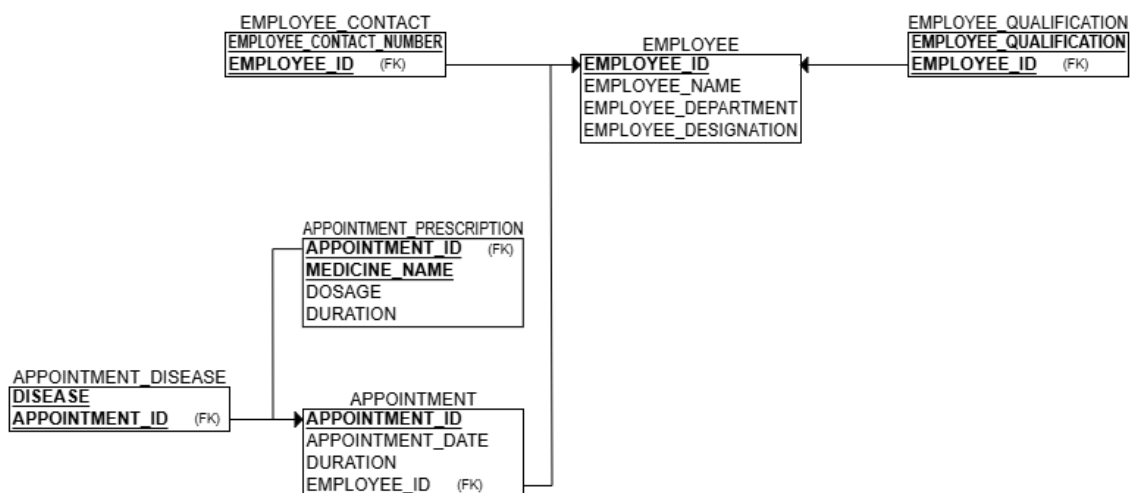
3.2.3 PATIENT AND APPOINTMENT



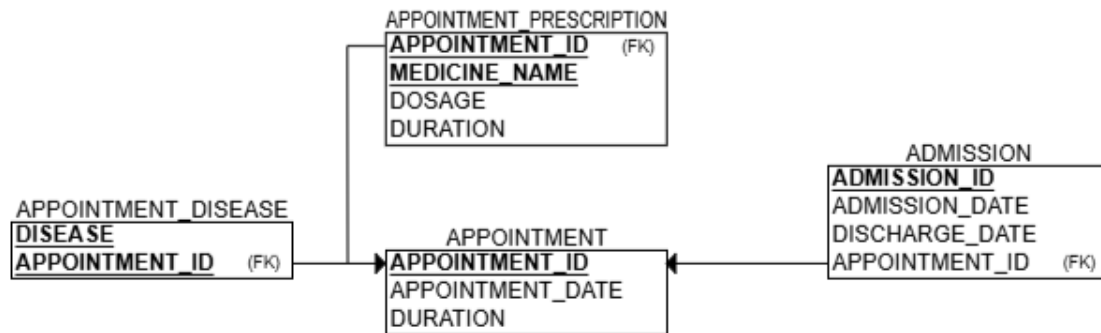
3.2.4 ADMITTED_PATIENT AND ADMISSION



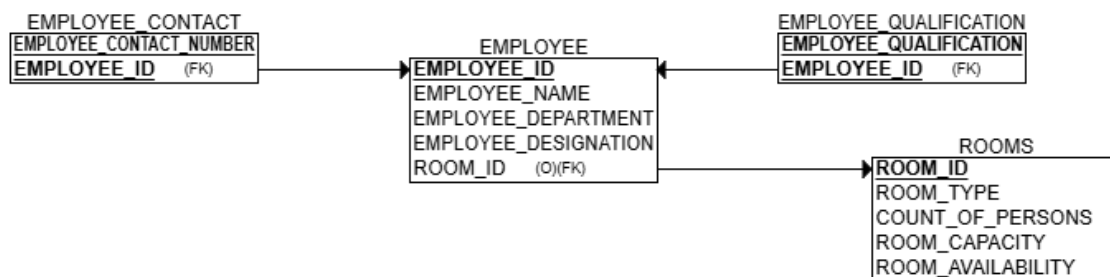
3.2.5 EMPLOYEE AND APPOINTMENT



3.2.6 APPOINTMENT AND ADMISSION



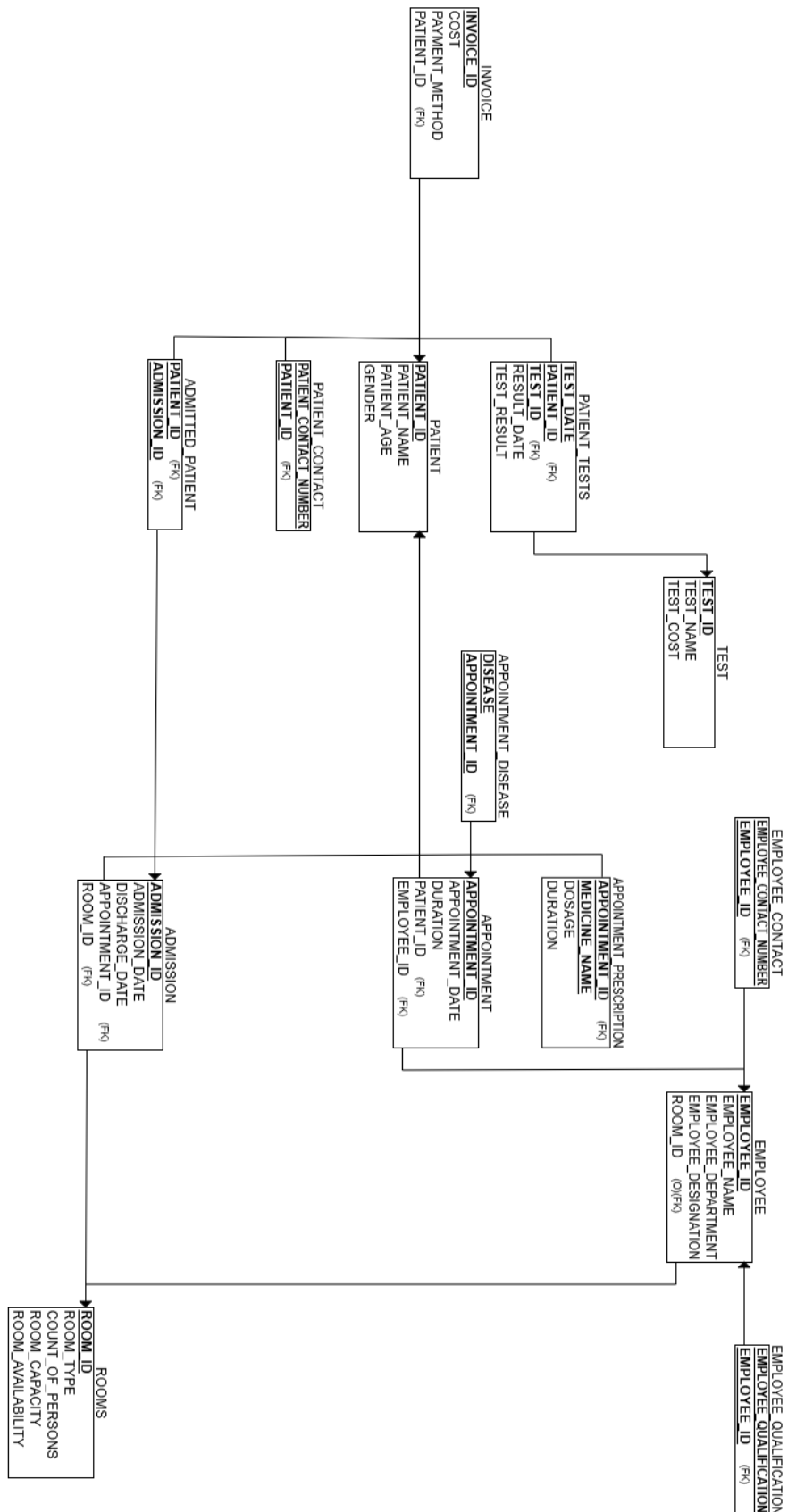
3.2.7 EMPLOYEE AND ROOMS



3.2.8 ADMISSION AND ROOMS



3.3 COMPLETE RELATIONAL SCHEMA



4. NORMALIZATION

In this project, the database design adheres to the principles of **Third Normal Form (3NF)**. As all entities and relationships have been structured to comply with 3NF, further normalization is not required. Each table is free of partial and transitive dependencies, ensuring that:

1. All attributes depend solely on the **primary key** (no partial dependency).
2. There are no transitive dependencies where non-prime attributes depend on other non-prime attributes.

By achieving 3NF, the schema ensures minimal redundancy, improved data integrity, and optimal design for efficient querying and maintenance.

5. SAMPLE DATA

This is the subset of the actual dataset in the .sql file which contains more records than this.

--INSERTING DATA INTO THE ROOMS TABLE

```
INSERT INTO ROOMS VALUES (201, 'DOCTOR ROOM', 0, 3, 'YES');
```

```
INSERT INTO ROOMS VALUES (202, 'DOCTOR ROOM', 0, 3, 'YES');
```

--INSERTING DATA INTO THE EMPLOYEE TABLE

```
INSERT INTO EMPLOYEE VALUES ('E001', 'Dr. Hasan', 'Cardiology', 'Senior Doctor', 201);
```

```
INSERT INTO EMPLOYEE VALUES ('E002', 'Dr. Sara', 'Neurology', 'Senior Doctor', 202);
```

--INSERTING DATA INTO THE EMPLOYEE_CONTACT TABLE

```
INSERT INTO EMPLOYEE_CONTACT VALUES ('E001', '03111234567');
```

```
INSERT INTO EMPLOYEE_CONTACT VALUES ('E002', '03112234567');
```

--INSERTING DATA INTO THE EMPLOYEE_QUALIFICATION TABLE

```
INSERT INTO EMPLOYEE_QUALIFICATION VALUES ('E001', 'MBBS');
```

```
INSERT INTO EMPLOYEE_QUALIFICATION VALUES ('E002', 'MBBS');
```

--INSERTING DATA INTO THE PATIENT TABLE

```
INSERT INTO PATIENT VALUES ('P001', 'Ahmed Ali', 45, 'M');
```

INSERT INTO PATIENT VALUES ('P002', 'Ayesha Khan', 30, 'F');

--INSERTING DATA INTO THE PATIENT_CONTACT TABLE

INSERT INTO PATIENT_CONTACT VALUES ('P001', '03001234567');

INSERT INTO PATIENT_CONTACT VALUES ('P002', '03002345678');

--INSERTING DATA INTO THE APPOINTMENT TABLE

INSERT INTO APPOINTMENT VALUES (1, 'E001', 'P001', SYSDATE);

INSERT INTO APPOINTMENT VALUES (2, 'E002', 'P002', SYSDATE);

--INSERTING DATA INTO THE APPOINTMENT_DISEASE TABLE

INSERT INTO APPOINTMENT_DISEASE VALUES (1, 'Hypertension');

INSERT INTO APPOINTMENT_DISEASE VALUES (2, 'Diabetes');

--INSERTING DATA INTO THE APPOINTMENT_PERScription TABLE

INSERT INTO APPOINTMENT_PERScription VALUES (1, 'Paracetamol', '500mg', '7 days');

INSERT INTO APPOINTMENT_PERScription VALUES (2, 'Amoxicillin', '250mg', '10 days');

--INSERTING DATA INTO THE ADMISSION TABLE

INSERT INTO ADMISSION (ADMISSION_ID, APPOINTMENT_ID, ROOM_ID,
ADMISSION_DATE) VALUES (101, 1, 203, SYSDATE);

INSERT INTO ADMISSION VALUES (103, 4, 205, SYSDATE, TO_DATE('2024-12-25', 'YYYY-MM-DD'));

--INSERTING DATA INTO THE TEST TABLE

INSERT INTO TEST VALUES (1, 'Blood Test', 500);

INSERT INTO TEST VALUES (2, 'X-Ray', 1500);

--INSERTING DATA INTO THE PATIENT_TESTS TABLE

INSERT INTO PATIENT_TESTS (PATIENT_ID, Test_ID, TEST_DATE) VALUES ('P001', 1, SYSDATE);

INSERT INTO PATIENT_TESTS VALUES ('P001', 2, SYSDATE, TO_DATE('2024-12-24', 'YYYY-MM-DD'), 'Negative');

--INSERTING DATA INTO THE INVOICE TABLE

INSERT INTO INVOICE VALUES (1001, 'P001', 1500.00, 'Credit Card');

INSERT INTO INVOICE VALUES (1002, 'P002', 2000.00, 'Cash');

6. SAMPLE QUERIES

6.1 GET EMPLOYEE INFORMATION

```
SELECT *  
  
FROM EMPLOYEE E, EMPLOYEE_CONTACT C, EMPLOYEE_QUALIFICATION Q  
  
WHERE E.EMPLOYEE_ID = C.EMPLOYEE_ID AND E.EMPLOYEE_ID = Q.EMPLOYEE_ID;
```

EMPLOYEE_ID	EMPLOYEE_NAME	EMPLOYEE_DEPARTMENT	EMPLOYEE_DESIGNATION	ROOM_ID	EMPLOYEE_ID	EMPLOYEE_CONTACT_NUMBER	EMPLOYEE_ID	QUALIFICATION
E001	Dr. Hasan	Cardiology	Senior Doctor	201	E001	03111234567	E001	MBBS
E002	Dr. Sara	Neurology	Senior Doctor	202	E002	03112234567	E002	MBBS
E003	Imran Khan	Helping Staff	Receptionist	-	E003	03113234567	E003	BSN
E004	Dr. Ali Akbar	Pediatrics	Junior Doctor	204	E004	03114234567	E004	MBBS
E005	Sana Yousaf	Helping Staff	Receptionist	-	E005	03115234567	E005	BSN
E006	Dr. Zafar Iqbal	Oncology	Surgeon	204	E006	03116234567	E006	MBBS
E006	Dr. Zafar Iqbal	Oncology	Surgeon	204	E006	03116234567	E006	MS
E007	Nadia Bashir	Medical Staff	Nurse	-	E007	03117234567	E007	GNM
E008	Khalid Waleed	Medical Staff	Nurse	-	E008	03118234567	E008	GNM
E009	Dr. Faisal	Cardiology	Senior Doctor	201	E009	03119234567	E009	MBBS

More than 10 rows available. Increase rows selector to view more rows.

6.2 GET APPOINTMENTS FOR A SPECIFIC PATIENT

```
SELECT A.APPOINTMENT_ID, A.APPOINTMENT_DATE, E.EMPLOYEE_NAME  
  
FROM APPOINTMENT A  
  
JOIN EMPLOYEE E ON A.EMPLOYEE_ID = E.EMPLOYEE_ID  
  
WHERE A.PATIENT_ID = 'P002';
```

APPOINTMENT_ID	APPOINTMENT_DATE	EMPLOYEE_NAME
2	18-DEC-24	Dr. Sara

1 rows returned in 0.01 seconds

[CSV Export](#)

6.3 GET ALL TESTS FOR A PATIENT

```
SELECT T.TEST_NAME, PT.TEST_DATE, PT.TEST_RESULT  
  
FROM PATIENT_TESTS PT  
  
JOIN TEST T ON PT.TEST_ID = T.TEST_ID  
  
WHERE PT.PATIENT_ID = 'P001';
```

TEST_NAME	TEST_DATE	TEST_RESULT
Blood Test	16-DEC-24	-
X-Ray	16-DEC-24	Negative

6.4 GET ALL EMPLOYEES IN A DEPARTMENT

```
SELECT EMPLOYEE_NAME, EMPLOYEE_DESIGNATION  
FROM EMPLOYEE  
WHERE EMPLOYEE_DEPARTMENT = 'Neurology';
```

EMPLOYEE_NAME	EMPLOYEE_DESIGNATION
Dr. Sara	Senior Doctor
Dr. Anum Akram	Surgeon

2 rows returned in 0.00 seconds

[CSV Export](#)

6.5 GET PATIENT INVOICE

```
SELECT I.INVOICE_ID, I.COST, I.PAYMENT_METHOD  
FROM INVOICE I  
WHERE I.PATIENT_ID = 'P001';
```

INVOICE_ID	COST	PAYMENT_METHOD
1001	1500	Credit Card

6.6 GET ALL PRESCRIPTIONS GIVEN BY A DOCTOR

```
SELECT A.APPOINTMENT_ID, D.DISEASE, P.MEDICINE_NAME, P.DOSAGE, P.DURATION  
FROM APPOINTMENT A  
JOIN APPOINTMENT_DISEASE D ON A.APPOINTMENT_ID = D.APPOINTMENT_ID  
JOIN APPOINTMENT_PERScription P ON A.APPOINTMENT_ID = P.APPOINTMENT_ID  
WHERE A.EMPLOYEE_ID = 'E001';
```

APPOINTMENT_ID	DISEASE	MEDICINE_NAME	DOSAGE	DURATION
1	Hypertension	Paracetamol	500mg	7 days
10	Migraine	Lisinopril	10mg	7 days

7 TABLE CREATION CODE/TRIGGERS/ROLES

7.1 TABLE CREATION

--DROP TABLES

DROP TABLE INVOICE;

DROP TABLE PATIENT_TESTS;

DROP TABLE TEST;

DROP TABLE ADMITTED_PATIENT;

DROP TABLE ADMISSION;

DROP TABLE APPOINTMENT_PERSSCRIPTION;

DROP TABLE APPOINTMENT_DISEASE;

DROP TABLE APPOINTMENT;

DROP TABLE PATIENT_CONTACT;

DROP TABLE PATIENT;

DROP TABLE EMPLOYEE_QUALIFICATION;

DROP TABLE EMPLOYEE_CONTACT;

DROP TABLE EMPLOYEE;

DROP TABLE ROOMS;

-- DROP ROLES/VIEWS

DROP ROLE PATIENT_ROLE;

DROP ROLE RECEPTIONIST;

DROP ROLE DOCTOR;

DROP ROLE NURSE;

DROP ROLE LAB_TECHNICIAN;

DROP VIEW DOCTOR_VIEW;

DROP VIEW PATIENT_VIEW;

DROP VIEW EMPLOYEE_VIEW;

DROP VIEW PATIENT_APPOINTMENT;

DROP VIEW NURSE_PATIENT;

-- CREATE TABLES

CREATE TABLE ROOMS

(ROOM_ID NUMBER NOT NULL,
ROOM_TYPE VARCHAR(15),
COUNT_OF_PERSONS NUMBER,
ROOM_CAPACITY NUMBER,
ROOM_AVAILABILITY VARCHAR(3) CHECK(ROOM_AVAILABILITY IN ('YES', 'NO')),
CONSTRAINT ROOMS_PK PRIMARY KEY (ROOM_ID));

CREATE TABLE TEST

(TEST_ID NUMBER NOT NULL,
TEST_NAME VARCHAR(25),
TEST_COST DECIMAL(6,2),
CONSTRAINT TEST_PK PRIMARY KEY (TEST_ID));

CREATE TABLE EMPLOYEE

(EMPLOYEE_ID VARCHAR(25) NOT NULL,
EMPLOYEE_NAME VARCHAR(25),
EMPLOYEE_DEPARTMENT VARCHAR(40),
EMPLOYEE_DESIGNATION VARCHAR(25),
ROOM_ID NUMBER,
CONSTRAINT EMPLOYEE_PK PRIMARY KEY (EMPLOYEE_ID),
CONSTRAINT EMPLOYEE_FK FOREIGN KEY (ROOM_ID) REFERENCES
ROOMS(ROOM_ID));

CREATE TABLE EMPLOYEE_CONTACT

```
(EMPLOYEE_ID VARCHAR(25) NOT NULL,  
EMPLOYEE_CONTACT_NUMBER VARCHAR(15) NOT NULL,  
CONSTRAINT EMPLOYEE_CONTACT_PK PRIMARY KEY (EMPLOYEE_ID,  
EMPLOYEE_CONTACT_NUMBER),  
CONSTRAINT EMPLOYEE_CONTACT_FK FOREIGN KEY (EMPLOYEE_ID) REFERENCES  
EMPLOYEE(EMPLOYEE_ID));
```

```
CREATE TABLE EMPLOYEE_QUALIFICATION  
(EMPLOYEE_ID VARCHAR(25) NOT NULL,  
QUALIFICATION VARCHAR(40) NOT NULL,  
CONSTRAINT EMPLOYEE_QUALIFICATION_PK PRIMARY KEY (EMPLOYEE_ID,  
QUALIFICATION),  
CONSTRAINT EMPLOYEE_QUALIFICATION_FK FOREIGN KEY (EMPLOYEE_ID)  
REFERENCES EMPLOYEE(EMPLOYEE_ID));
```

```
CREATE TABLE PATIENT  
(PATIENT_ID VARCHAR(25) NOT NULL,  
PATIENT_NAME VARCHAR(25),  
PATIENT_AGE NUMBER,  
GENDER VARCHAR(1) CHECK(GENDER IN ('M', 'F')),  
CONSTRAINT PATIENT_PK PRIMARY KEY (PATIENT_ID));
```

```
CREATE TABLE PATIENT_CONTACT  
(PATIENT_ID VARCHAR(25) NOT NULL,  
PATIENT_CONTACT_NUMBER VARCHAR(15) NOT NULL,  
CONSTRAINT PATIENT_CONTACT_PK PRIMARY KEY (PATIENT_ID,  
PATIENT_CONTACT_NUMBER),  
CONSTRAINT PATIENT_CONTACT_FK FOREIGN KEY (PATIENT_ID) REFERENCES  
PATIENT(PATIENT_ID));
```

```
CREATE TABLE APPOINTMENT
```

```
(APPOINTMENT_ID NUMBER NOT NULL,  
EMPLOYEE_ID VARCHAR(25) NOT NULL,  
PATIENT_ID VARCHAR(25),  
APPOINTMENT_DATE DATE DEFAULT SYSDATE,  
CONSTRAINT APPOINTMENT_PK PRIMARY KEY (APPOINTMENT_ID),  
CONSTRAINT APPOINTMENT_FK1 FOREIGN KEY (EMPLOYEE_ID) REFERENCES  
EMPLOYEE(EMPLOYEE_ID),  
CONSTRAINT APPOINTMENT_FK2 FOREIGN KEY (PATIENT_ID) REFERENCES  
PATIENT(PATIENT_ID));
```

```
CREATE TABLE APPOINTMENT_DISEASE  
(APPOINTMENT_ID NUMBER NOT NULL,  
DISEASE VARCHAR(25) NOT NULL,  
CONSTRAINT APPOINTMENT_DISEASE_PK PRIMARY KEY (APPOINTMENT_ID,  
DISEASE),  
CONSTRAINT APPOINTMENT_DISEASE_FK FOREIGN KEY (APPOINTMENT_ID)  
REFERENCES APPOINTMENT(APPOINTMENT_ID));
```

```
CREATE TABLE APPOINTMENT_PERScription  
(APPOINTMENT_ID NUMBER NOT NULL,  
MEDICINE_NAME VARCHAR(25) NOT NULL,  
DOSAGE VARCHAR(40),  
DURATION VARCHAR(25),  
CONSTRAINT APPOINTMENT_PERScription_PK PRIMARY KEY (APPOINTMENT_ID,  
MEDICINE_NAME),  
CONSTRAINT APPOINTMENT_PERScription_FK FOREIGN KEY (APPOINTMENT_ID)  
REFERENCES APPOINTMENT(APPOINTMENT_ID));
```

```
CREATE TABLE ADMISSION  
(ADMISSION_ID NUMBER NOT NULL,  
APPOINTMENT_ID NUMBER,
```

```

ROOM_ID NUMBER,
ADMISSION_DATE DATE DEFAULT SYSDATE,
DISCHARGE_DATE DATE,
CONSTRAINT ADMISSION_PK PRIMARY KEY (ADMISSION_ID),
CONSTRAINT ADMISSION_FK1 FOREIGN KEY (APPOINTMENT_ID) REFERENCES
APPOINTMENT(APPOINTMENT_ID),
CONSTRAINT ADMISSION_FK2 FOREIGN KEY (ROOM_ID) REFERENCES
ROOMS(ROOM_ID));

CREATE TABLE ADMITTED_PATIENT
(PATIENT_ID VARCHAR(25) NOT NULL,
ADMISSION_ID NUMBER NOT NULL,
CONSTRAINT ADMITTED_PATIENT_PK PRIMARY KEY (PATIENT_ID, ADMISSION_ID),
CONSTRAINT ADMITTED_PATIENT_FK1 FOREIGN KEY (PATIENT_ID) REFERENCES
PATIENT(PATIENT_ID),
CONSTRAINT ADMITTED_PATIENT_FK2 FOREIGN KEY (ADMISSION_ID) REFERENCES
ADMISSION(ADMISSION_ID));

CREATE TABLE PATIENT_TESTS
(PATIENT_ID VARCHAR(25) NOT NULL,
TEST_ID NUMBER NOT NULL,
TEST_DATE DATE DEFAULT SYSDATE NOT NULL,
RESULT_DATE DATE,
TEST_RESULT VARCHAR(15),
CONSTRAINT PATIENT_TESTS_PK PRIMARY KEY (PATIENT_ID, TEST_ID, TEST_DATE),
CONSTRAINT PATIENT_TESTS_FK1 FOREIGN KEY (PATIENT_ID) REFERENCES
PATIENT(PATIENT_ID),
CONSTRAINT PATIENT_TESTS_FK2 FOREIGN KEY (TEST_ID) REFERENCES
TEST(TEST_ID));

CREATE TABLE INVOICE

```

```
(INVOICE_ID NUMBER NOT NULL,  
PATIENT_ID VARCHAR(25),  
COST DECIMAL(10,2),  
PAYMENT_METHOD VARCHAR(25),  
CONSTRAINT INVOICE_PK PRIMARY KEY (INVOICE_ID),  
CONSTRAINT INVOICE_FK FOREIGN KEY (PATIENT_ID) REFERENCES  
PATIENT(PATIENT_ID));
```

7.2 TRIGGERS

CREATE OR REPLACE TRIGGER ADMISSION_INSERTION_TRIGGER

AFTER INSERT ON ADMISSION

FOR EACH ROW

DECLARE

v_patient_id VARCHAR(25);

BEGIN

SELECT PATIENT_ID

INTO v_patient_id

FROM APPOINTMENT

WHERE APPOINTMENT_ID = :NEW.APPOINTMENT_ID;

-- Insert into ADMITTED_PATIENT table

INSERT INTO ADMITTED_PATIENT (PATIENT_ID, ADMISSION_ID)

VALUES (v_patient_id, :NEW.ADMISSION_ID);

END;

/

CREATE OR REPLACE TRIGGER CHECK_EMPLOYEE_DOCTOR

BEFORE INSERT ON APPOINTMENT

FOR EACH ROW

DECLARE

v_employee_name EMPLOYEE.EMPLOYEE_NAME%TYPE;

BEGIN

SELECT EMPLOYEE_NAME

INTO v_employee_name

FROM EMPLOYEE

WHERE EMPLOYEE_ID = :NEW.EMPLOYEE_ID;


```

    IF v_employee_name NOT LIKE 'Dr.%' THEN

        RAISE_APPLICATION_ERROR(-20001, 'Employee must be a doctor (name starts with
"Dr.")');

    END IF;

END;

/

```

```

CREATE OR REPLACE TRIGGER ROOMS_ADMISSION_INSERTION
BEFORE INSERT OR DELETE ON ADMISSION
FOR EACH ROW

```

```

DECLARE

```

```

    v_room_capacity ROOMS.ROOM_CAPACITY%TYPE;

```

```

    v_current_count ROOMS.COUNT_OF_PERSONS%TYPE;

```

```

BEGIN

```

```

    -- Handle Insert Operation

```

```

    IF INSERTING THEN

```

```

        IF :NEW.ROOM_ID IS NOT NULL THEN

```

```

            SELECT COUNT_OF_PERSONS, ROOM_CAPACITY

```

```

            INTO v_current_count, v_room_capacity

```

```

            FROM ROOMS

```

```

            WHERE ROOM_ID = :NEW.ROOM_ID;

```

```

        IF NVL(v_current_count, 0) >= v_room_capacity THEN

```

```

            RAISE_APPLICATION_ERROR(-20001, 'Cannot insert: Room capacity has been
reached.');
```

```

        ELSE

```

```

            UPDATE ROOMS

```

```

            SET COUNT_OF_PERSONS = NVL(COUNT_OF_PERSONS, 0) + 1

```

```

            WHERE ROOM_ID = :NEW.ROOM_ID;

```

```

        END IF;
    END IF;

    -- Handle Delete Operation
    ELSIF DELETING THEN
        IF :OLD.ROOM_ID IS NOT NULL THEN
            SELECT COUNT_OF_PERSONS
            INTO v_current_count
            FROM ROOMS
            WHERE ROOM_ID = :OLD.ROOM_ID;

            IF NVL(v_current_count, 0) <= 0 THEN
                RAISE_APPLICATION_ERROR(-20002, 'Cannot delete: No persons in the room.');
            ELSE
                UPDATE ROOMS
                SET COUNT_OF_PERSONS = NVL(COUNT_OF_PERSONS, 0) - 1
                WHERE ROOM_ID = :OLD.ROOM_ID;
            END IF;
        END IF;
    END IF;
END;

/

CREATE OR REPLACE TRIGGER ROOMS_EMPLOYEE_INSERTION
BEFORE INSERT OR DELETE ON EMPLOYEE
FOR EACH ROW
DECLARE
    v_room_capacity ROOMS.ROOM_CAPACITY%TYPE;
    v_current_count ROOMS.COUNT_OF_PERSONS%TYPE;

```

```

BEGIN

-- Handle Insert Operation

IF INSERTING THEN

    IF :NEW.ROOM_ID IS NOT NULL THEN

        SELECT COUNT_OF_PERSONS, ROOM_CAPACITY
        INTO v_current_count, v_room_capacity
        FROM ROOMS
        WHERE ROOM_ID = :NEW.ROOM_ID;

        IF NVL(v_current_count, 0) >= v_room_capacity THEN

            RAISE_APPLICATION_ERROR(-20003, 'Cannot insert: Room capacity has been
reached.');
```

```

        ELSE

            UPDATE ROOMS

            SET COUNT_OF_PERSONS = NVL(COUNT_OF_PERSONS, 0) + 1

            WHERE ROOM_ID = :NEW.ROOM_ID;

        END IF;

    END IF;

-- Handle Delete Operation

ELSIF DELETING THEN

    IF :OLD.ROOM_ID IS NOT NULL THEN

        SELECT COUNT_OF_PERSONS
        INTO v_current_count
        FROM ROOMS
        WHERE ROOM_ID = :OLD.ROOM_ID;

        IF NVL(v_current_count, 0) <= 0 THEN

            RAISE_APPLICATION_ERROR(-20004, 'Cannot delete: No persons in the room.');
```

```
ELSE
    UPDATE ROOMS
    SET COUNT_OF_PERSONS = NVL(COUNT_OF_PERSONS, 0) - 1
    WHERE ROOM_ID = :OLD.ROOM_ID;
END IF;
END IF;
END IF;
END;
/
```

7.3 ROLES CREATION

--CREATE ROLES/ VIEWS

--CREATING VIEWS FOR PATIENT

CREATE VIEW DOCTOR_VIEW AS

SELECT E.EMPLOYEE_NAME, E.EMPLOYEE_DEPARTMENT, E.EMPLOYEE_DESIGNATION,
Q.QUALIFICATION, E.ROOM_ID

FROM EMPLOYEE E, EMPLOYEE_QUALIFICATION Q

WHERE E.EMPLOYEE_ID = Q.EMPLOYEE_ID

AND EMPLOYEE_NAME LIKE 'Dr%';

CREATE VIEW PATIENT_VIEW AS

SELECT P.PATIENT_ID, P.PATIENT_NAME, P.PATIENT_AGE, P.GENDER,
C.PATIENT_CONTACT_NUMBER

FROM PATIENT P, PATIENT_CONTACT C

WHERE P.PATIENT_ID = C.PATIENT_ID;

--CREATING ROLE PATIENT AND PRIVILEGING PERMISSIONS

CREATE ROLE PATIENT_ROLE;

GRANT SELECT, INSERT, UPDATE (PATIENT_NAME, PATIENT_AGE, GENDER,
PATIENT_CONTACT_NUMBER) ON PATIENT_VIEW TO PATIENT_ROLE;

GRANT SELECT ON TEST TO PATIENT_ROLE;

GRANT SELECT ON DOCTOR_VIEW TO PATIENT_ROLE;

--CREATING VIEW FOR RECEPTIONIST

CREATE VIEW EMPLOYEE_VIEW AS

SELECT E.EMPLOYEE_NAME, E.EMPLOYEE_DEPARTMENT, E.EMPLOYEE_DESIGNATION,
Q.QUALIFICATION, C.EMPLOYEE_CONTACT_NUMBER, E.ROOM_ID

FROM EMPLOYEE E, EMPLOYEE_CONTACT C, EMPLOYEE_QUALIFICATION Q

WHERE E.EMPLOYEE_ID = Q.EMPLOYEE_ID AND E.EMPLOYEE_ID = C.EMPLOYEE_ID;

--CREATING ROLE FOR RECEPTIONIST

CREATE ROLE RECEPTIONIST;

GRANT INSERT, SELECT, UPDATE ON PATIENT TO RECEPTIONIST;

GRANT SELECT ON EMPLOYEE_VIEW TO RECEPTIONIST;

GRANT SELECT ON ROOMS TO RECEPTIONIST;

GRANT SELECT, INSERT, UPDATE, DELETE ON APPOINTMENT TO RECEPTIONIST;

GRANT SELECT ON TEST TO RECEPTIONIST;

GRANT INSERT, SELECT, UPDATE ON ADMISSION TO RECEPTIONIST;

GRANT INSERT, SELECT, UPDATE ON INVOICE TO RECEPTIONIST;

CREATE VIEW PATIENT_APPOINTMENT AS

SELECT A.PATIENT_ID, A.APPOINTMENT_DATE, D.DISEASE, P.MEDICINE_NAME, P.DOSAGE,
P.DURATION

FROM APPOINTMENT A, APPOINTMENT_DISEASE D, APPOINTMENT_PERScription P

WHERE A.APPOINTMENT_ID = D.APPOINTMENT_ID AND A.APPOINTMENT_ID =
P.APPOINTMENT_ID;

--CREATING ROLE FOR DOCTOR

CREATE ROLE DOCTOR;

GRANT SELECT, INSERT, UPDATE ON PATIENT_APPOINTMENT TO DOCTOR;

--CREATING VIEW FOR NURSE

CREATE VIEW NURSE_PATIENT AS

SELECT A.PATIENT_ID, AD.ADMISSION_DATE, D.DISEASE, P.MEDICINE_NAME, P.DOSAGE,
P.DURATION

FROM APPOINTMENT A, APPOINTMENT_DISEASE D, APPOINTMENT_PERScription P,
ADMISSION AD

WHERE A.APPOINTMENT_ID = D.APPOINTMENT_ID AND A.APPOINTMENT_ID =
P.APPOINTMENT_ID AND A.APPOINTMENT_ID = AD.APPOINTMENT_ID;

CREATE ROLE NURSE;

GRANT SELECT ON NURSE_PATIENT TO NURSE;

CREATE ROLE LAB_TECHNICIAN;

GRANT SELECT, INSERT, UPDATE ON PATIENT_TESTS TO LAB_TECHNICIAN;

GRANT SELECT ON TEST TO LAB_TECHNICIAN;

8. APPROXIMATED STORAGE

FIELD NAME	BYTES PER RECORD	APPROX RECORDS	STORAGE IN BYTES
ROOMS	27	100	2700
TEST	32	200	6400
EMPLOYEE	118	500	59,000
EMPLOYEE_CONTACT	40	1000	40,000
EMPLOYEE_QUALIFICATION	65	1500	97,500
PATIENT	54	10,000	540,000
PATIENT_CONTACT	46	20,000	920,000
APPOINTMENT	61	99,999	6,099,939
APPOINTMENT_DISEASE	29	299,997	8,699,913
APPOINTMENT PRESCRIPTION	94	499,995	46,999,530
ADMISSION	25	50,000	1,250,000
ADMITTED_PATIENT	29	50,000	1,450,000
PATIENT_TESTS	57	30,000	1,710,000
INVOICE	60	30,000	1,800,000